velpTEC
EDUTAINMENT

Projektarbeit

Python



Ausgangssituation:

In einer Ära, in der Informationen im Überfluss vorhanden sind, ist die Fähigkeit, relevante und prägnante Zusammenfassungen von Texten zu erstellen, von entscheidender Bedeutung. In diesem einwöchigen Projekt werden wir uns darauf konzentrieren, ein Python-Programm zu entwickeln, das mithilfe der Natural Language Toolkit (NLTK) die Technik des extraktiven Textzusammenfassens implementiert.

Unser Ziel ist es, eine Softwarelösung zu entwickeln, die in der Lage ist, aus umfangreichen Texten die wichtigsten Informationen zu extrahieren und in einem prägnanten Zusammenfassungstext darzustellen. Dazu werden wir uns mit den Grundlagen der Textverarbeitung und den Funktionen des NLTK vertraut machen, um effektive Extraktionsalgorithmen zu implementieren.

Oben haben wir bereits erwähnt, dass es sich bei dem Projekt um eine extraktive Textzusammenfassung handeln wird. Im Allgemeinen unterscheidet man zwischen zwei verschiedenen Arten von Textzusammenfassungen: extraktive und abstraktive.

Extraktive Zusammenfassungen:

Basierend auf vorhandenen Textpassagen:
Extraktive Zusammenfassungen wählen relevante Sätze oder Abschnitte aus dem Originaltext aus und fassen diese zusammen. Es werden also keine neuen Formulierungen oder Konzepte eingeführt.

Einfachheit und Direktheit:
Da extraktive Zusammenfassungen nur vorhandene Textteile verwenden, sind sie oft einfacher und direkter zu implementieren. Sie erfordern weniger komplexe Textgenerationsalgorithmen.

Bewahrung von Originalität und Kontext:
Da die ausgewählten Textpassagen direkt aus dem Originaltext stammen, bewahren extraktive Zusammenfassungen die Originalität und den Kontext des Textes weitgehend.

Abstraktive Zusammenfassungen:

Generierung neuer Formulierungen:
Im Gegensatz zu extraktiven Zusammenfassungen können abstraktive Zusammenfassungen neue Formulierungen und Konzepte einführen, die nicht im Originaltext enthalten sind. Dies erfordert oft komplexere Sprachverarbeitungstechniken wie Natural Language Generation (NLG).

Kreativität und Flexibilität:
Abstraktive Zusammenfassungen sind flexibler und können kreativer sein, da sie nicht auf vorhandene Textpassagen beschränkt sind. Sie können alternative Formulierungen verwenden, um den gleichen Inhalt auszudrücken.

Herausforderungen bei der Generierung:
Die Generierung neuer Texte kann jedoch herausfordernd sein, da sie ein tieferes Verständnis des Inhalts erfordert und oft dazu neigt, grammatikalische und semantische Fehler zu produzieren.

Vergleich:

Extraktive Zusammenfassungen sind oft einfacher zu implementieren und bewahren die Originalität des Textes, während abstraktive Zusammenfassungen flexibler und kreativer sind, aber komplexere Verarbeitungstechniken erfordern.
Extraktive Zusammenfassungen eignen sich gut für die schnelle Erfassung der Hauptpunkte eines Textes, während abstraktive Zusammenfassungen

nützlicher sein können, wenn es darum geht, den Inhalt auf innovative Weise zu präsentieren oder wenn der Originaltext nicht präzise genug ist.
Die Wahl zwischen extraktiven und abstraktiven Zusammenfassungen hängt von verschiedenen Faktoren ab, darunter der Anwendungsfall, die Verfügbarkeit von Trainingsdaten und die gewünschte Genauigkeit und Kreativität der Zusammenfassung.

In dieser Projektarbeit werden wir ein extraktives Textzusammenfassungsprogramm erstellen. Dafür bekommst du einen Text über Climate Science zur Verfügung gestellt.

Ein extraktives Textzusammenfassungsprogramm lässt sich in den folgenden Schritten erstellen:

1. Tokenisiere den Text
2. Entferne Satzzeichen ("?",".","!",",",";",":","'","-","_") und Stop-Words
3. Zähle wie oft die übrig gebliebenen Worte im Text vorkommen und normiere dies mit dem maximalen Wert
4. Teile den Text in einzelne Sätze auf
5. Bestimme den Word Count für jeden Satz.
6. Wähle aus den Sätzen die 20% der Sätze aus, die den höchsten Word Count haben.

Deine Aufgabenpakete:

1. Mach dich mit der **nltk - Bibliothek** vertraut
   Installiere die nltk Bibliothek in deiner Entwicklungsumgebung. Finde heraus, mit welcher nltk-Funktion du einen Text tokenisieren kannst. Die Tokenisierung von Texten ist der Prozess der Aufteilung eines zusammenhängenden Textes in kleinere Einheiten, die als "Token" bezeichnet werden, wie z.B. Wörter, Satzzeichen oder spezielle Zeichenfolgen. Dieser Prozess erleichtert die spätere Verarbeitung und Analyse des Textes, da er es ermöglicht, einzelne Bestandteile des Textes getrennt zu behandeln.
   Finde heraus, mit welcher nltk-Funktion du einen Text in Sätze aufteilen kannst.
   Finde heraus, wie du die in der nltk-Bibliothek integrierten Stopwords verwenden kannst. Stopwords sind Wörter, die in einem Text sehr häufig vorkommen und wenig semantische Bedeutung tragen, wie z.B. "und", "oder" und "aber". Sie werden oft aus Texten entfernt, um die Verarbeitung zu beschleunigen und die Genauigkeit von Textanalysen zu verbessern.

2. **Tokenisiere** den Text
   Schreibe eine Funktion, die den Text tokenisiert. Achte darauf, dass du Satzzeichen und Stopwords entfernst. Am Ende solltest du eine Liste mit den wichtigsten Worten haben, die im Text vorkommen.

3. **Zähle** die übrigen Worte
   Nachdem du den Text tokenisiert hast, zähle nun wie oft jedes Wort im Text vorkommt (der Word Count). Es bietet sich hier an, mit Dictionaries zu arbeiten. Nachdem du gezählt hast, wie oft jedes Wort vorkommt, identifiziere das Wort mit der größten Häufigkeit. Danach normierst du die übrigen Word Counts mit diesem maximalen Word

Count.

4. **Sentence Count** für jeden Satz
   Bestimme für jeden Satz, der im Text vorkommt, den Sentence Count. Das bedeutet, du summierst den Word Count von jedem vorkommenden Wort (sofern es in der Tokenliste ist) auf. Hier ein kleines Beispiel:

   Unser Dictionary mit Word Counts ist:
   {“Climate”: 3 , “Science”: 1 , “Weather”: 5 , “Cars”: 2}
   Der Satz, dessen Sentence Count wir bestimmen wollen, sieht beispielsweise so aus:
   “Cars have a huge influence on Climate Change.”
   Der Sentence Count ist dann:
   WordCount(Climate) + WordCount(Cars) = 3 + 2 = 5

5. **Extraktive** Textzusammenfassung
   Jetzt hast du für jeden Satz im Text einen Sentence Count erstellt. Nun wählst du die 20% der Sätze mit dem größten Sentence Count aus. Diese fungieren nun als Zusammenfasssung für den gesamten Text. Die finale Textzusammenfassung soll als String im Terminal/Ausgabekonsole ausgegeben werden.

Anforderungen:

1. Es ist von großer Bedeutung, deinen Code sorgfältig zu dokumentieren, um eine klare Verständlichkeit und Wartbarkeit sicherzustellen. Verwende hierfür Docstrings, um den Zweck und die Funktionalität deiner Funktionen und Module zu beschreiben. Klare Kommentare innerhalb des Codes helfen anderen Entwicklern und dir selbst, den Code besser zu verstehen. Nutze aussagekräftige Variablennamen, die den Zweck und die Verwendung der Variablen deutlich machen, und vermeide kryptische Abkürzungen.

2. Es ist ebenso wichtig, unnötigen Code zu vermeiden und den Code schlank zu halten. Entferne alles, was nicht unbedingt erforderlich ist, um die Funktionalität zu erfüllen. Das vereinfacht nicht nur die Wartung und Fehlerbehebung, sondern verbessert auch die Lesbarkeit des Codes.

3. Darüber hinaus solltest du bei der Entwicklung deines Codes auf Modularität und Erweiterbarkeit achten. Dein Code sollte so strukturiert sein, dass er problemlos erweitert werden kann, um neuen Anforderungen gerecht zu werden. Durch eine modulare Architektur wird die Wiederverwendbarkeit erhöht und die Entwicklung neuer Funktionen erleichtert. Dies ermöglicht es dir und anderen Entwicklern, auf bereits vorhandenem Code aufzubauen und weitere Entwicklungen vorzunehmen, ohne dabei die Gesamtstruktur des Projekts grundlegend ändern zu müssen